Axona

Architecture

David A. Smith
Axona.net

An Axona Architecture Note v0.4.2 2026-05-26

A protocol is the contract between layers that don't trust each other.

What I cannot create, I do not understand.
Richard Feynman · 1988

I The Stack

Axona ships as three repositories that compose into one running system. Each layer has a single responsibility,[1] a single canonical version identifier (`KERNEL_VERSION`), and a contract surface narrow enough to fit on a postcard.

`@axona/protocol` (kernel). The pure-JS library. Owns the DHT routing, the pub/sub overlay, the cryptography, and the transport contracts. No browser DOM, no Node-specific I/O, no network code outside abstract transport classes. Runs identically in browsers, in Node, in dht-sim's simulated network.

`axona-peer` (browser application). The reference end-user app. Provides identity persistence, region selection, WebRTC mesh formation, a chat-style UI for pub/sub. Imports the kernel as a vendored copy under `vendor/axona-protocol/` so the version that ran when the page was built is the version that runs in the browser.

`axona-bridge` (deployed server). The signaling + introducer service at `bridge.axona.net`. Runs an embedded `AxonaPeer` of its own (so it's a full DHT participant, not just a relay), gates connecting peers behind a version handshake, relays WebRTC signaling between browsers, and forwards Axona wire frames addressed to its own embedded peer.

[1] The pattern is intentional. A peer's business logic should never need to know the wire format; a bridge should never need to know what topic a subscriber cares about; the kernel should never need to know which application is using it. Every interface in this document is a place where one layer was given the right to be wrong about the layer below.

The three components carry their own semver tracks because each ships to a different audience: kernel changes need API-level discipline; peer changes are user-visible UI; bridge changes are operational deploys. But they all share the same kernel version, visible at runtime through `/healthz` (bridge), the `welcome` frame (any peer connecting), and the version row in the peer UI.

A fourth component, `dht-sim`, runs the kernel against a simulated network for protocol research and regression benchmarks. It's not part of the production stack but shares the same `@axona/protocol` library; every change that ships to production also has to pass the simulator's 25,000-node battery.

II Identity and Addressing

Every Axona participant has a 264-bit identifier. The top 8 bits are an S2 geographic cell[2] computed from the peer's latitude and longitude; the bottom 256 bits are the SHA-256 of the peer's Ed25519 public key. A peer in London has an ID starting with `0x4f`; a peer

[2] S2 is Google's hierarchical cell decomposition of the sphere. The top 8 bits split the world into 192 cells of roughly 800 km radius each; an S2 prefix gives Axona a region-aware address without leaking GPS-level precision.

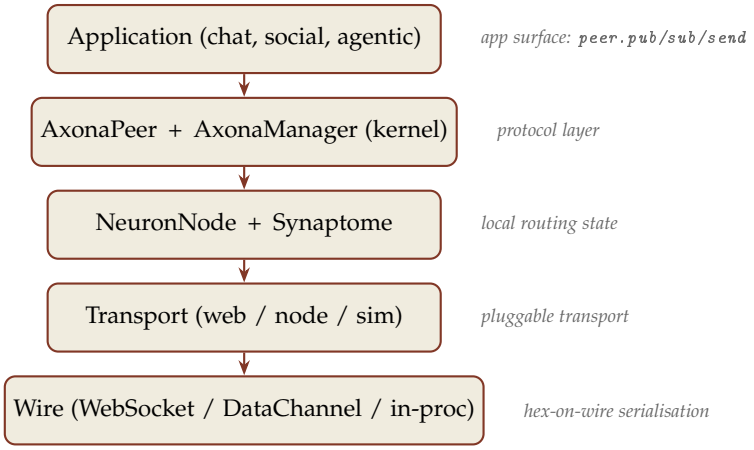

Figure 1: * Five layers in one peer. Every arrow is a contract; every contract has a smoke-test suite.

in Virginia starts with `0xdf`. Two peers in the same region share a leading byte; two random peers share only the bits their keys happen to share.

The address space is treated as `BigInt` in every line of running code. The wire format is 66-character lowercase hex — the ONLY representation that ever appears in JSON. The conversion happens at exactly two places: when the transport serialises an outgoing frame, and when the dispatcher parses an incoming one.[3]

Topic identifiers share the address space. A topic is named by the application (a string like `us-east/hello-world`) and optionally anchored at a publisher: `topicId = sha256(publisher || topic-name)`. When the publisher is null the topic is global (top byte `00`); when it's a regional synthetic publisher (top byte = S2 cell, bottom bits zero), the topic inherits the region. That's what makes regional pub/sub route naturally: peers in the same region are XOR-closest to topics anchored at that region.

The Activation Potential (AP) routing formula, the K-closest selector, and the synaptome's stratum buckets all operate on this one space. There is no separate "topic ID space" or "peer ID space"; they share addresses because they share routing.

III The Kernel: `@axona/protocol`

THE KERNEL IS THE PROTOCOL. Everything else is its clients. The directory structure mirrors the contract surface:

`src/contracts/`. Abstract base classes: `Transport`, `DHT`, `DHTNode`, `BootstrapService`, `PersistenceAdapter`. These are the seams

[3] This rule didn't exist in versions 1.0 through 1.4. The kernel mixed hex strings and BigInts in different layers and the boundaries weren't enforced; pub/sub silently failed because `transport.notify(bigint)` returned without sending against a hex-keyed Map. v1.5.0 made BigInt-internal canonical; `_normPeerId` helpers and `typeof v === 'bigint'` defensive branches are gone.

where alternate implementations plug in. A custom transport (libp2p, QUIC, embedded) extends `Transport`; a database-backed persistence (IndexedDB, SQLite) extends `PersistenceAdapter`.

`src/dht/`. The active routing layer: `NeuronNode`, `Synapse`, `AxonaPeer`, `AxonaDomain`, `Subscription`. This is where the Axona routing logic lives, and where the developer-facing API surface (`peer.pub`, `peer.sub`, `peer.lookup`) is declared.

`src/pubsub/`. The pub/sub overlay: `AxonaManager`, `post.js` (topic ID derivation), `envelope.js` (signed publish envelopes), `ed25519.js` (Ed25519 binding for `crypto.subtle`). AxonaManager owns the K-closest tree, the replay caches, the dedup LRU, the metrics counters.

`src/transport/`. Concrete transports: `transport/web/` (browser: BridgeTransport over WebSocket + WebRTCTransport over DataChannels), `transport/node/` (server-side WebSocket), `transport/sim/` (in-process simulation for dht-sim and tests). All implement the abstract `Transport` contract identically.

`src/utils/`. Pure helpers: `hexid.js` (BigInt↔hex conversions, XOR distance, leading-zero count), `s2.js` (S2 cell computation), JSON codec (`transport/wire.js`).

`src/identity/`. Ed25519 keypair derivation + persistence envelope. The single entry point `deriveIdentity` takes a `{lat, lng}` or full options object and returns `{id, pubkey, pubkeyHex, privateKey, region, ...}`.

The whole kernel has zero runtime dependencies beyond the host JavaScript environment (browser or Node 20+). No bundler, no transpilation, no build step. The same source files run in the browser via ES modules, in Node via direct `import`, and in the simulator via the in-process transport.

IV The Local Node

Each peer's view of the network lives in two data structures on the `NeuronNode`: the *synaptome* (its outgoing connections) and the *incoming synapse index* (peers known to point at it). Both are keyed by 264-bit BigInt nodeId.

A `Synapse` is a routing edge — it carries a peer ID, a measured latency, a learned weight, a stratum (the XOR-distance bucket), an inertia lock, and provenance (was this synapse added via handshake,

hop-cache, triadic introduction, or sponsor bootstrap?). The synaptome is unbounded by design; the routing layer evicts synapses through the *Structural Survival Rule*, which guarantees navigability even under aggressive weight decay.

The rule, in one sentence: a synapse may only be pruned if at least one other synapse with the same stratum exists. The effect: every populated stratum keeps at least one live route, so the address space stays globally navigable even after heavy decay.

Routing decisions don't pick the XOR-closest synapse blindly. Each candidate is scored by Activation Potential:

$$AP_c \;=\; \frac{\Delta\text{Distance}_c}{L_c} \times (1 + w \cdot W_c)$$

The first factor is progress velocity — how much XOR distance the hop closes per millisecond of measured latency. The second is a mild boost for synapses that have historically led to fast lookups. Weight only accrues on at-or-below-average-latency paths, so a high-W synapse genuinely represents a fast route, not just a frequently-used one. Two random hops with similar latency tie; the one that's served us well before wins the tiebreak.

V The Protocol Layer

AxonaPeer is the active object that turns a `NeuronNode` into a network participant. It owns:

- The handler installation. On `peer.start()` it registers the Axona routing handlers (`lookup_step`, `lookahead_probe`, `find_closest_set`, `route_msg`, `local_probe`, `triadic_introduce`, `hop_cache`, `lateral_spread`, `reinforce`) on the transport.
- The auto-admission loop. It asks `transport.boundPeers()` for currently-bound nodeIds and seeds them into the synaptome; it subscribes to `transport.onPeerBound` for future admissions.
- The pub/sub surface. `peer.pub`, `peer.sub`, `peer.pull`, `peer.metrics`: the developer-facing API. Each delegates to the AxonaManager after deriving the topic ID and (for `pub`) building a signed envelope.
- The direct-message surface. `peer.send`, `peer.notify`, `peer.onMessage`: per-peer request/response and fire-and-forget, outside the pub/sub overlay. For protocols that aren't topic-shaped.
- The lookup primitive. `peer.lookup(targetKey)` runs an iterative K-closest walk and returns `{found, path, hops, peerId, latency}`. This is what AxonaManager uses internally on every `pub/sub` to find the topic's axons.

The `AxonaPeer` surface is described in detail in the companion document *AxonaPeer Overview* (v0.4.0). This section is the architectural placement; the field-by-field description lives there.

`AxonaManager` is a separate object owned by the peer. Its job is the pub/sub overlay: when a subscriber sends `pubsub:subscribe-k` to the K-closest axons for a topic, the receiving AxonaManager stores

the subscriber in its `role.children` for that topic. When a publisher fans `pubsub:publish-k` to its own K-closest, each receiving AxonaManager iterates its `role.children` and forwards `pubsub:deliver` to each subscriber. The publisher's signed envelope flows through unchanged; per-hop dedup prevents the fan-out tree from delivering duplicates.

The K-closest replication is what makes the system tolerant. A publisher sends to *five* root axons in parallel; a subscriber subscribes at *five* root axons in parallel. As long as one axon in each set overlaps, delivery succeeds. With $K = 5$ and synaptomes that converge under the bridge's peer-list dissemination, overlap is typically 4 or 5 of 5 — so a publish that loses two axons to transient WebRTC failures still delivers cleanly.

VI The Transport Layer

Transport is where addresses meet sockets. The kernel's `Transport` contract is a small surface: `start`, `stop`, `send` (request/response), `notify` (fire-and-forget), `isConnected`, `bindPeer`, `boundPeers`, `onPeerBound`, `onRequest`, `onNotification`, `onPeerDied`. Every concrete transport implements this surface and the upper layers don't care which one is wired in.

Three concrete implementations ship:

`transport/web/`. The browser stack. A `CompositeTransport` fans out to two sub-transports based on which one owns each nodeId: a `BridgeTransport` that carries Axona wire frames over the same WebSocket the bridge uses for signaling, and a `WebRTCTransport` over a `MeshManager` that owns the data-channel pool. The factory `webTransport({bridgeUrl, identity})` composes the two, drives the bridge's version gate, runs the hello/hello-ack admission, threads TURN credentials from the welcome frame into the mesh's ICE config, and resolves once everything is up.

`transport/node/`. A Node-side WebSocket transport. Used by the bridge's embedded peer to talk to connected browsers without simulating WebRTC — the bridge *is* the signaling server, so it doesn't need WebRTC to reach any browser; the WebSocket is direct.

`transport/sim/`. The simulator transport. Used by dht-sim's `TransportAxonaEngine` and by the kernel's smoke tests. Identical Transport contract, but messages are scheduled in the event loop with simulated latency derived from a real internet topology dataset.

The wire format is JSON, with a single quirk: BigInt is serialised as `"<digits>n"` (decimal digits with an `n` suffix) and revived back to BigInt on decode. NodeId fields are always hex strings on the wire — they never appear as bigint-suffixed literals because the kernel converts them at frame construction.

The same kernel test suite runs against all three transports. Cross-transport bugs — which is to say, the kind of bug where a fix lands in one transport's call path but not the others — can't ship.

VII The Bridge

The bridge is an introducer, not a server. Its job is to let browsers find each other; once they've found each other, WebRTC data channels carry every byte of application traffic peer-to-peer. The bridge stays in the picture only for two reasons: (1) it's the rendezvous for the version handshake before peers can talk at all; (2) it's a guaranteed-reachable mesh peer, which makes it a useful K-closest axon in small networks where two browsers might otherwise fail to find common axons.

The bridge handles three kinds of frames over the WebSocket:

Signaling. `welcome`, `peer-list`, `peer-joined`, `peer-left`, `signal`, `ping/pong`. Routes WebRTC SDP offers/answers and ICE candidates between browsers, broadcasts join/leave events, supplies short-lived TURN credentials in the welcome frame.

Version handshake. `client-hello/ server-hello`. Enforces a minimum peer version. Rejects sub-minimum peers with WebSocket close code 4426 and a human-readable reason; the peer's UI shows a "please reload" banner.

Axona wire. `{type: 'axona', payload: ...}` frames carrying `req/res/ntf` messages between any connected browser and the bridge's embedded peer. These are the same frames any two browsers exchange over WebRTC — the bridge embeds a full `AxonaPeer` of its own (`BridgeAxonaNode`) and participates in routing just like any other node, which is why it can be in K-closest sets and why it can hold subscriber children for pub/sub topics.

The bridge's embedded peer has a fixed identity (cf3138... in production, anchored at London). Its `health` surface exposes the kernel version it's running and the synaptome it has built up, available at `/healthz` JSON. Operators check that endpoint to confirm a deploy.

VIII The Wire Protocol

Every frame on the wire is JSON. Every nodeId or topic ID field is a 66-character lowercase hex string. There's no binary framing, no protobuf, no length-prefixed records — the discipline of having one wire format that any of the three transports can carry is worth the few percent of size you give up.

Three kinds of message-typed envelopes flow over the wire:

Request *(`k: 'req'`).* A correlated request expecting a response. Carries an integer `id` for correlation, a `type` string naming the handler (`lookup_step`, `route_msg`, `find_closest_set`, ...), and an opaque `body`.

Response *(`k: 'res'`).* The reply to a `req`. Echoes the `id` and adds an `ok` boolean and a `body`.

Notification *(`k: 'ntf'`).* Fire-and-forget. No `id`, no response. Used for hello/hello-ack admission, `pubsub:subscribe-k`, `pubsub:publish-k`, `pubsub:deliver`, `triadic_introduce`, `hop_cache`, `lateral_spread`, `reinforce`.

The handler types are versioned implicitly through the kernel — the bridge's `minPeerVersion` gate excludes peers running an older protocol that wouldn't understand newer messages. Major protocol changes bump `KERNEL_VERSION`'s minor; minor changes are additive (a peer that doesn't know a handler type silently drops the message).

A pub/sub message envelope is signed:

```
{
  msgId:        "<sha256 of canonical(envelope) hex>",
  ts:           1716638400000,
  topic:        "us-east/hello-world",
  message:      <application payload>,
  signerPubkey: "<Ed25519 pubkey hex>",
  signature:    "ed25519:<128 hex chars>"
}
```

Receivers verify the signature against the signer's pubkey using `crypto.subtle.verify`; the kernel surfaces verification result on the delivered envelope so applications can decide whether to trust unsigned or unverifiable publishes.

IX Routing

Axona's routing is learning-adaptive. It replaces Kademlia's k-bucket array with a dynamic synaptome that adjusts edge weights based on observed latency. The basic shape — iterative lookup, $\log_2(N)$ hop count — is unchanged from Kademlia; what changes is which peer you ask at each hop.

The five wire operations that drive Axona routing are:

lookup_step (req). "Here's a target. What's your best next hop?" The receiver evaluates AP across its synaptome's progress candidates, returns the winning synapse + its own routing state.

lookahead_probe (req). "If I forward to you for target T, who would you forward to?" A one-deep peek that lets the caller compare expected hop costs across candidates before committing.

find_closest_set (req). "Give me your K peers closest to this target." The K-closest harvester used by AxonaManager's pub/sub axon discovery.

route_msg (req). Generic routed-message forwarder. Used by AxonaManager for `__tunneled_direct__` delivery when the target isn't directly bound.

reinforce (ntf). "This lookup succeeded; bump the weight on the synapse I used." Fire-and-forget LTP signal back along the path. Only flows when the lookup hit at-or-below average latency for the path's region.

Two structural-plasticity messages let the network rewire itself without explicit instruction: `triadic_introduce` ("introduce me to your friend at X") and `hop_cache` ("I've forwarded to X several times; install a direct synapse to X"). Both are fire-and-forget notifications that obey the synaptome's admission rule (`_addByVitality`).

The performance claim of Axona over Kademlia is roughly 40% fewer wall-clock milliseconds per lookup at 25,000 nodes, mostly from latency-weighted hop selection. The figure is documented in the companion paper *Axona: A Learning-Adaptive DHT*.

X Pub/Sub: K-Closest

The pub/sub overlay is its own structure. It sits on top of Axona routing but doesn't reuse the synaptome for state — each peer maintains a separate set of *axon roles*, one per topic it's chosen as an axon for. A role holds a topic ID, a set of children (subscribers), a replay cache, and a K-replicated peer set (the other root axons for the same topic).

Subscriber-side flow:

1. Compute `topicId = sha256(publisher || topic-name)`.
2. Find the *K* peers closest to `topicId` in the local synaptome. (No network walk — the local reachable peer set is what matters; remote-walk K-closest returns ghost IDs that route fan-out into dead ends.)
3. Send `pubsub:subscribe-k` to each, carrying the subscriber's nodeId and an optional `lastSeenTs` for replay.
4. Each receiver runs `_addOrRecruitChild`, stores the subscriber under `role.children`, and optionally serves a `pubsub:replay-batch` of cached messages newer than `lastSeenTs`.

Publisher-side flow:

1. Compute the same `topicId`.
2. Build a signed envelope.
3. Find the same *K* peers closest to `topicId` locally.
4. Send `pubsub:publish-k` to each.
5. Each receiver runs `_onPublishDirect`: dedup against `_seenPublishes`, add to replay cache, fan `pubsub:deliver` to every child in `role.children`.
6. Each child receives `pubsub:deliver`, dedups, fires the local `_deliveryCallback` which routes to the application's subscription handler.

When a publisher's K-closest peers also happen to be in the subscribers' K-closest sets, the tree converges and delivery is single-hop from publisher to axon to subscriber. When they don't, the system relies on the redundancy: a missing axon's subtree is covered by the four other axons. The Activation Potential weighting makes high-quality axons dominate the K-closest set over time.

Per-hop `_alreadySeenPublish` dedup is keyed on `publishId` (a string like `nodeId:counter`), separate from the content-hash `msgId` the application sees. Multiple arrivals of the same `publishId` at one peer collapse to a single `_deliveryCallback` firing.

XI The Application API

Every Axona application talks to the protocol through one import. The barrel export at `@axona/protocol` is the entire contract surface — a small set of factory functions, one peer class, and a handful of helpers. This section is the field-by-field reference.

Identity

`deriveIdentity({ lat, lng })` *generate a fresh peer identity*

`lat` number. Latitude in decimal degrees.

lng number. Longitude in decimal degrees.

Returns a Promise resolving to `{ id, pubkey, pubkeyHex, privkey, privateKey, region }`. `id` is the 66-char hex `nodeId` (8-bit S2 prefix from `(lat,lng)` + 256-bit SHA-256 of the public key). `privateKey` is the in-memory `CryptoKey` ready for signing; `privkey` is the base64-encoded export for persistence. The whole envelope is what `AxonaPeer` expects as `identity`.

`dumpIdentity(identity)` *serialize for storage*

Returns a Promise resolving to a plain JSON object you can write to disk or IndexedDB. The reciprocal `loadIdentity(envelope)` reconstructs the `CryptoKey`-bearing form on the next launch.

Transports

`webTransport({ ... })` *browser transport: WebSocket to the bridge + WebRTC mesh*

bridgeUrl string. e.g. `wss://bridge.axona.net`.

identity identity envelope from `deriveIdentity`.

log (optional, default `() => {}`*)* function `(event, data)` that receives transport-internal diagnostic events.

WebSocketImpl (optional) constructor for the `WebSocket` class. Defaults to `globalThis.WebSocket`.

autoHandshake (optional, default `true`*)* when true, `transport.start()` drives the full bridge admission sequence end-to-end — the WebSocket version gate, the application-level `hello`/`hello-ack` exchange, and binding the bridge as a peer. Set to false only if you're driving the handshake yourself.

peerVersion (optional, default `KERNEL_VERSION`*)* semver string sent in `client-hello`.

handshakeTimeoutMs (optional, default `15000`*)* how long to wait for the bridge's `hello` before rejecting `start()`.

Returns a `CompositeTransport`. After `await transport.start(identity.id)` the bridge is reachable as a peer (`transport.bridgeNodeId`) and the WebRTC mesh is ready to accept channels.

`nodeTransport.server({ ... })` *server-side WebSocket transport (Node 20+)*

`nodeTransport.client({ ... })` *Node WebSocket client*

Used by `axona-bridge`'s embedded peer and by Node applications. Same `Transport` contract as the browser transport; no WebRTC.

`simTransport({ network, identity })` *in-process simulation transport*

network `SimNetwork` shared across peers.

identity identity envelope.

For tests and `dht-sim`. Messages are scheduled in the event loop with synthetic latency from a real internet topology dataset.

AxonaPeer

`new AxonaPeer({ ... })` *the per-node DHT instance*

node `NeuronNode` instance for this peer (carries id, lat, lng, the synaptome).

domain `AxonaDomain` the peer joins. Construct one per network with `new AxonaDomain({ k: 20 })`, where `k` *(default 20)* is the *K*-replication factor.

transport the transport instance from `webTransport`, `nodeTransport`, or `simTransport`.

identity (optional) identity envelope. Required for signed publishes (the default). Apps that only call `peer.pub(t, m, { sign: false })` can omit it.

axonaManager (optional) explicit `AxonaManager`. When omitted, one is built automatically on `peer.start()`.

persist (optional) `PersistenceAdapter` for synaptome / subscription / replay-cache checkpoints. Default: no persistence.

Lifecycle

`await peer.start()` *wire handlers, install delivery hook*
`await peer.stop()` *detach handlers and event listeners*
`await peer.join(sponsor?)` *production bootstrap path*

sponsor (optional) 66-char hex node ID. If given, opens a transport channel to that peer and seeds the synaptome from it. If omitted, returns immediately — the peer is "joined" but isolated; inbound connections from other peers populate the synaptome.

`await peer.leave(opts?)` *leave the network gracefully*

opts.drain (default `true`*)* wait for in-flight publishes to settle.

opts.notify (default `true`*)* send a `peer-leaving` notification to every peer in the synaptome.

opts.timeoutMs (default `5000`*)* maximum drain window in ms.

Pub / Sub

`await peer.pub(topic, message, opts?)` *publish to a topic*

`topic` string. Application-level topic name (any non-empty string).

`message` JSON-serializable payload.

`opts.sign (default true)` sign the envelope with the identity's Ed25519 key.

`opts.publisher (default: this peer's nodeId)` controls where the topic is anchored in the DHT. `undefined` means this peer publishes its own topic; `null` means the public well-known mode (`topicId` = SHA256(`topic`), anyone can publish); a 66-char hex string lets a peer relay under that publisher's anchor (rare).

Returns a Promise resolving to the `msgId` — the 64-char hex content hash of the envelope. Throws `PublishError` on invalid inputs or signing failure.

`await peer.sub(topic, handler, opts?)` *subscribe to a topic*

`topic` string.

`handler` function `(envelope) => void` invoked for each delivery. The envelope is `{ msgId, ts, topic, message, publisher, signerPubkey }`.

`opts.publisher (default: this peer's nodeId)` same semantics as `pub` — must match the publisher's `publisher` value for the subscription to land at the right topic ID.

`opts.since (default: live tail)` replay control. Pass `'latest'` for the most-recent cached message plus future deliveries, `'all'` for the full replay cache plus future, or a Unix-ms `number` for everything newer than that timestamp.

Returns a Promise resolving to a `Subscription` handle. Call `sub.unsubscribe()` to stop receiving.

`await peer.pull(msgId, opts)` *fetch a single message by ID from the relay's replay cache*

`msgId` 64-char hex content hash.

`opts.topic` string. The topic name (required, used to locate the relay).

`opts.publisher` 66-char hex publisher ID, or `null` for public topics.

`opts.timeoutMs (default 1000)`.

Returns the envelope or `null` if not found.

`await peer.metrics(topic, opts)` *aggregate counters across the relay tree*

topic string.

opts.publisher 66-char hex publisher ID, or `null`.

opts.timeoutMs *(default* `500`*).*

Returns `{ publishes, subscribers, deliveries, pulls, reshares, relayCount }`.

Direct messaging

`await peer.send(targetId, message)` *RPC-style; awaits the remote handler's return value*
`peer.notify(targetId, message)` *fire-and-forget*
`peer.onMessage(handler)` *register a handler for incoming direct messages*

targetId 66-char hex node ID. The peer must already be in the synaptome.

message JSON-serializable payload.

handler function `(fromId, message) => result`. The return value is sent back to the caller on `send`; ignored on `notify`. Async handlers are awaited.

Mesh introspection

`peer.peers()` returns an array of 66-char hex node IDs currently in the synaptome.
`peer.onPeerJoin(handler)` registers `(nodeId) => void` for new arrivals.
`peer.onPeerLeave(handler)` registers `(nodeId) => void` for departures.
`peer.health()` returns `{ started, transportConnected, synapseCount, subscriptionCount, kernelVersion }`.
`peer.onLog(level, handler)` `level` is `'info'`, `'warn'`, or `'error'`.
`peer.onError(handler)` registers `(err) => void` for unhandled errors.

Lookup

`await peer.lookup(targetKey)` *iterative K-closest walk*

targetKey 66-char hex 264-bit key. Usually a topic ID or a node ID.

Returns `{ found, path, hops, peerId, latency }`. `found` is the closest reachable peer; `path` is the list of nodes traversed; `hops` is the count; `latency` is wall-clock ms.

Snapshots

`await peer.snapshot()` returns a JSON object capturing identity, synaptome, axon roles, subscriptions, replay caches. Symmetric with `AxonaPeer.fromSnapshot(snap, deps)`, which reconstructs a peer from one of these dumps without touching the network.

Helpers

`await deriveTopicId(publisher, topic)` returns the 66-char hex topic ID. `publisher` may be `null` (public mode) or a 66-char hex node ID. Useful for displaying the publish location in a UI.

`geoCellId(lat, lng, bits)` returns the integer S2 cell at the given precision. `bits = 8` matches the top 8 bits of every Axona nodeId.

XII Identity and Security

Identity is an Ed25519 keypair. The kernel uses `crypto.subtle.generateKey` to produce a curve-Ed25519 pair; the public key is hashed with SHA-256 to produce the bottom 256 bits of the nodeId; the top 8 bits are the S2 cell of the user's `(lat, lng)`. The private key never leaves the host environment, but the public key is broadcast on every connection (in hello/hello-ack) so peers can verify each other.

Every signed publish carries the publisher's public key (`signerPubkey`) inline. The kernel verifies the `ed25519:<hex>` signature against `signerPubkey` using `crypto.subtle.verify`. Verification is per-envelope, not per-peer — a peer that's untrusted can still relay signed publishes from a trusted publisher unchanged, and any receiver along the way can verify the chain of custody.

Persistence is opt-in. The kernel's `PersistenceAdapter` contract exposes `write(namespace, value)` and `read(namespace)`; concrete adapters target IndexedDB (browser) and the filesystem (Node). When wired, AxonaPeer checkpoints identity, synaptome seeds, and pending subscriptions on a 5-second debounce. On restart, the identity envelope is reloaded and validated (the stored nodeId must match the freshly-derived one); subscriptions are reinstalled as soon as the mesh has stabilised.

The `crypto.subtle` dependency is a hard requirement: the demo and the peer fail to boot in insecure contexts (HTTP pages outside localhost) because the Web Crypto API simply isn't available. The bridge serves over HTTPS; the peer's GitHub Pages deployment enforces HTTPS at the Pages level; the demo at `demo.axona.net`

Because identity is bound to a region prefix, moving geographic location changes the nodeId. axona-peer's identity.js preserves the existing identity across reloads unless the user explicitly re-selects a different region. A roaming user keeps the same DHT identity for the life of the browser profile.

required Let's Encrypt cert provisioning before identity derivation would run at all.

XIII Production Deployment

The public stack runs at three URLs.

bridge.axona.net. The deployed bridge. Runs on a DigitalOcean droplet, started by systemd, kept current via `git pull && npm ci -omit=dev && systemctl restart`. Exposes `/healthz` for monitoring. Listens on `wss://` only; HTTP requests are redirected.

axona.net. The reference peer application, served via GitHub Pages from the `axona-peer` repo's main branch. Every push redeploys. The vendored `axona-protocol` kernel ships with the app; the version row in the UI confirms which kernel an open tab is running.

demo.axona.net. The minimal pub/sub demo, served via GitHub Pages from the `axona-protocol` repo's main branch (where `examples/minimal-pubsub-browser/` lives). Same Pages deployment pattern; HTTPS-enforced because `crypto.subtle` requires a secure context.

A coordinated release works as follows: a fix lands in `axona-protocol`, bumps `KERNEL_VERSION` and the package version, tags the commit. `axona-peer` resyncs its `vendor/axona-protocol/` from the new kernel, bumps `PEER_VERSION`, and pushes (Pages picks up). The bridge updates its `@axona/protocol` dependency in `package.json`, bumps `VERSION` in `server.js`, pushes, and the operator runs the deploy script. Every component ends up reporting the same `kernelVersion` in its visible metadata.

Every Axona component is required to surface its `KERNEL_VERSION` at a runtime-inspectable point: the bridge in `/healthz` and the welcome frame, the peer in its UI version row, the demo in its footer. The rule exists because before it did, "which kernel is everyone on?" required SSH-ing into the bridge and grepping `node_modules`.

XIV Future Directions

The architecture is stable. The shape of the layering, the wire format, the BigInt-canonical address discipline, the K-closest pub/sub semantics — these are not expected to change. What's coming sits inside the existing seams:

Federated bridges. Multiple bridges in different geographic regions, gossiping peer-lists so a peer connected to one bridge can be reached from another. Single-bridge today is a deployment simplification, not a protocol constraint.

Persistent topics. Today's replay cache survives in-memory only. A pluggable persistence layer for axon `role.replayCache` (with a TTL) lets late subscribers catch up across bridge restarts.

Topic access control. Public-mode topics (`publisher: null`) let anyone publish. Private topics (`publisher: <fixed-pubkey>`) accept publishes only from that key. A capability layer where subscribers verify per-publish signatures against an allowed-signer set lives at the application layer today; lifting it into the kernel is straightforward.

Cross-tab state sharing. Multiple tabs on the same browser today derive the same identity and form independent peers; a BroadcastChannel-based shared peer would reduce mesh load proportional to tab count.

Native transport. A C-based or Rust-based embedded kernel for IoT-class devices, sharing the wire format with the JS kernel so existing peers can talk to it without adaptation.

The job of the architecture is to make these additions land cleanly. The job of the existing layers is to keep working while they do.

Companion documents: NeuronNode Overview *(v0.4.0)*, AxonaPeer Overview *(v0.4.0)*, Axona Explainer *(v0.4.25), and the source repositories at* `github.com/axona-net`.

A Appendix: The Demo Application

This is the complete source of the minimal-pubsub demo deployed at `demo.axona.net`. The file lives in the kernel repository under `examples/minimal-pubsub-browser/index.html`. Three hundred and seventy-six lines of HTML and JavaScript, no build step, no dependencies outside the kernel barrel export. Open it in two tabs; one publishes, the other receives. The structure maps to the API reference in §XI section by section: identity, transport, peer construction, lifecycle, subscribe, publish.

examples/minimal-pubsub-browser/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>@axona/protocol -- webTransport pub/sub demo</title>
  <style>
    :root {
      --rust: #2d7373;  --rust-soft: #d4a896;
```

```
  --ink: #1a1a1a;   --muted: #666;
  --bg: #fffff8;    --card-bg: #fbfbf6;
  --border: #d8d4ca; --code-bg: #f0ecdf;
  --green: #2d7373; --red: #c0392b; --amber: #b07d3a;
}
* { box-sizing: border-box; }
body {
  font-family: 'EB Garamond', Georgia, serif;
  max-width: 1100px; margin: 0 auto;
  padding: 2rem 1.5rem;
  background: var(--bg); color: var(--ink); line-height: 1.5;
}
h1 { font-weight: 500; letter-spacing: -0.015em; margin-bottom: 0.3em; }
.lede { color: var(--muted); margin-top: 0; }
code { font-family: 'Inconsolata', ui-monospace, Menlo, monospace;
       background: var(--code-bg); padding: 1px 4px;
       border-radius: 2px; font-size: 0.92em; }

.status { background: var(--card-bg); border: 1px solid var(--border);
          border-radius: 6px; padding: 0.6rem 0.9rem; margin-top: 1rem;
          font-size: 14px; display: flex; align-items: center; gap: 0.7rem; }
.dot { width: 10px; height: 10px; border-radius: 50%; background: var(--muted);
  }
.dot.connecting { background: var(--amber); }
.dot.connected  { background: var(--green); }
.dot.failed     { background: var(--red); }
.status-text { flex: 1; }
.status-text code { font-size: 12px; }

.panes { display: grid; grid-template-columns: 1fr 1fr;
         gap: 1.2rem; margin-top: 1.2rem; }
.pane { background: var(--card-bg); border: 1px solid var(--border);
        border-radius: 8px; padding: 1rem 1.2rem; }
.pane h2 { font-size: 18px; margin: 0 0 0.5rem 0;
           color: var(--rust); font-weight: 600; }
.label { color: var(--muted); font-size: 13px; margin: 0.7rem 0 0.3rem 0; }
.nodeId { font-family: 'Inconsolata', ui-monospace, monospace;
          font-size: 11px; color: var(--muted); word-break: break-all;
          padding: 0.3rem 0.5rem; background: var(--code-bg); border-radius: 3
 px; }
.row { display: flex; gap: 0.5rem; margin-bottom: 0.5rem; }
input[type=text] { padding: 0.5rem 0.7rem; border: 1px solid var(--border);
                   border-radius: 4px; font-family: inherit; font-size: 15px;
                   background: white; width: 100%; }
.row input[type=text] { flex: 1; }
button { background: var(--rust); color: white; border: none;
         padding: 0.5rem 1rem; border-radius: 4px; cursor: pointer;
         font-family: inherit; font-size: 15px; font-weight: 500;
         white-space: nowrap; }
button:disabled { background: #aaa; cursor: not-allowed; }
.received { background: white; border: 1px solid var(--border);
            border-radius: 4px; padding: 0.6rem 0.9rem;
            font-family: 'Inconsolata', ui-monospace, monospace;
            font-size: 12.5px; min-height: 260px; max-height: 420px;
            overflow-y: auto; white-space: pre-wrap; color: #333; }
.received .empty { color: var(--muted); font-style: italic; }
.arrived { color: var(--rust); font-weight: 600; }
.self    { color: var(--rust-soft); font-style: italic; margin-left: 0.5em; }

.bootlog { background: var(--card-bg); border: 1px solid var(--border);
           border-radius: 6px; padding: 0.6rem 0.9rem;
           font-family: 'Inconsolata', ui-monospace, Menlo, monospace;
           font-size: 12px; color: var(--muted); margin-top: 1.5rem;
           min-height: 1.5rem; max-height: 260px; overflow-y: auto;
           white-space: pre-wrap; }
.bootlog .heading { color: var(--rust); font-weight: 600;
                    letter-spacing: 0.05em; text-transform: uppercase;
                    font-size: 11px; margin-bottom: 0.4em; }
```

```
    footer { margin-top: 1.5rem; font-size: 13px; color: var(--muted);
             text-align: center; }
    footer a { color: var(--rust); }
  </style>
</head>
<body>
  <h1>@axona/protocol -- pub/sub demo on the live bridge</h1>
  <p class="lede">
    This tab is an Axona peer connected to <code>wss://bridge.axona.net</code> via
    <code>webTransport</code>. Open this page in multiple tabs (or share the URL
    with a friend) and every connected peer publishing to
    <code>hello-world</code> shows up in the received list.
  </p>

  <div class="status">
    <span class="dot" id="dot"></span>
    <span class="status-text" id="statusText">connecting...</span>
  </div>

  <div class="panes">
    <div class="pane">
      <h2>Publish</h2>
      <div class="label">Your name</div>
      <input type="text" id="nameInput" value="Alice" />
      <div class="label">Message</div>
      <div class="row">
        <input type="text" id="msgInput" placeholder="message"
               value="hello from alice" />
        <button id="pubBtn" disabled>Publish</button>
      </div>
      <div class="label">Topic ID (publish location -- deterministic
        from publisher + topic name; MUST match on every peer)</div>
      <div class="nodeId" id="topicIdDisplay">...</div>
      <div class="label">Last published msgId (per-message content hash
        -- changes every publish)</div>
      <div class="nodeId" id="lastMsgId">--</div>
      <div class="label">Your nodeId</div>
      <div class="nodeId" id="myId">...</div>
      <div class="label">Your location (top 8 bits of nodeId = S2 cell
        of this lat/lng)</div>
      <div class="nodeId" id="myLocation">...</div>
      <div class="label">Synaptome (peers known to your DHT)</div>
      <div class="nodeId" id="synSize">...</div>
      <div class="label">Bridge</div>
      <div class="nodeId" id="bridgeUrl">...</div>
    </div>

    <div class="pane">
      <h2>Subscribe -- <code>hello-world</code></h2>
      <div class="label">Subscribed to topic ID (must match publisher's)</div>
      <div class="nodeId" id="subTopicId">...</div>
      <div class="received" id="receivedLog">
        <span class="empty">(waiting for messages...)</span>
      </div>
    </div>
  </div>

  <div class="bootlog">
    <div class="heading">console</div>
    <div id="bootlog">Booting...</div>
  </div>

  <footer>
    Source: <code>axona-protocol/src/</code> served live -- kernel
    v<span id="kernelVer">...</span> -- <code>webTransport({ bridgeUrl })</code>
    + <code>peer.pub</code>/<code>peer.sub</code>
  </footer>
```

```
<script type="module">
  import {
    AxonaPeer, AxonaDomain, NeuronNode,
    deriveIdentity, deriveTopicId, geoCellId,
  } from '/src/index.js';
  import { webTransport } from '/src/transport/web/index.js';

  // -- Bridge URL: ?bridge= override, default to the live bridge --
  function getBridgeUrl() {
    const fromQs = new URLSearchParams(location.search).get('bridge');
    if (fromQs) return fromQs;
    return 'wss://bridge.axona.net';
  }
  const BRIDGE_URL = getBridgeUrl();
  document.getElementById('bridgeUrl').textContent = BRIDGE_URL;

  const bootlogEl = document.getElementById('bootlog');
  const log = (msg, extra) => {
    const line = extra != null
      ? `${msg}  ${typeof extra === 'object' ? JSON.stringify(extra) : extra}`
      : msg;
    bootlogEl.textContent += '\n' + line;
    bootlogEl.scrollTop = bootlogEl.scrollHeight;
    console.log('[demo]', msg, extra ?? '');
  };
  bootlogEl.textContent = 'Booting...';

  const dotEl    = document.getElementById('dot');
  const statusEl = document.getElementById('statusText');
  function setStatus(state, text) {
    dotEl.className = 'dot ' + state;
    statusEl.textContent = text;
  }
  setStatus('connecting', `connecting to ${BRIDGE_URL}...`);

  // Pull kernel version for footer
  try {
    const pkg = await fetch('/package.json').then(r => r.json());
    document.getElementById('kernelVer').textContent = pkg.version;
  } catch { document.getElementById('kernelVer').textContent = '?'; }

  // -- 1. Geolocation ------------------------------------------------
  // The top 8 bits of every Axona nodeId are the S2 cell of the peer's
  // lat/lng, so peers in the same region are naturally XOR-closest to
  // topics anchored at that region.  Ask the browser; fall back to
  // (0,0) if denied.
  function getLocation() {
    const FALLBACK = { lat: 0, lng: 0, source: 'fallback (Gulf of Guinea)' };
    if (!navigator.geolocation) return Promise.resolve(FALLBACK);
    return new Promise(resolve => {
      navigator.geolocation.getCurrentPosition(
        pos => resolve({
          lat: pos.coords.latitude,
          lng: pos.coords.longitude,
          source: 'browser geolocation',
        }),
        err => { log('geolocation denied', { reason: err.message });
                 resolve(FALLBACK); },
        { enableHighAccuracy: false, timeout: 8000, maximumAge: 600_000 },
      );
    });
  }

  const location = await getLocation();
  document.getElementById('myLocation').textContent =
    `${location.lat.toFixed(4)}, ${location.lng.toFixed(4)} -- ${location.source
   }`;
```

```
log('location resolved', location);

// Regional synthetic publisher: nodeId-shaped hash whose top 8 bits
// are the S2 cell of this lat/lng and whose bottom 256 bits are zero.
function regionSynthPublisher({ lat, lng }) {
  const s2 = geoCellId(lat, lng, 8);
  return s2.toString(16).padStart(2, '0') + '0'.repeat(64);
}

// -- 2. Identity (264-bit Ed25519, anchored at the resolved location)
const identity = await deriveIdentity({
  lat: location.lat, lng: location.lng });
document.getElementById('myId').textContent = identity.id;
log('identity derived', { id: identity.id.slice(0, 16) + '...' });

// -- 3. Transport + peer ----------------------------------------
// webTransport's autoHandshake (default) drives the bridge version
// gate AND the application-level hello/hello-ack admission as part
// of start().  AxonaPeer.start() then asks the transport for its
// boundPeers() and auto-admits them into the synaptome.
const transport = webTransport({
  bridgeUrl: BRIDGE_URL,
  identity,
  log: (evt, extra) => log('wt:' + evt, extra),
});

const node = new NeuronNode({
  id:  BigInt('0x' + identity.id),
  lat: location.lat,
  lng: location.lng,
});
node.transport = transport;
const domain = new AxonaDomain({ k: 20 });
const peer = new AxonaPeer({ domain, node, identity, transport });

await transport.start(identity.id);   // does the bridge handshake
await peer.start();                   // auto-admits boundPeers()

window.__axona = { peer, transport, identity, node };
log('peer ready -- bridge:', transport.bridgeNodeId.slice(0, 16) + '...');
setStatus('connected',
  `connected -- bridge ${transport.bridgeNodeId.slice(0, 8)}...`);

// -- 4. Topic ID + live synaptome size ---------------------------
// The topic ID is the DETERMINISTIC publish location in the DHT.
// For this hello-world demo we use a GLOBAL topic (publisher = null
// -> topicId = sha256(topicName)), so anyone running the demo
// anywhere in the world lands at the same topicId and can chat.
const TOPIC     = 'hello-world';
const publisher = null;   // global public topic
const topicId   = await deriveTopicId(publisher, TOPIC);
document.getElementById('topicIdDisplay').textContent = topicId;
log('topicId derived',
    { topicId, publisher, topicName: TOPIC });

const synSizeEl = document.getElementById('synSize');
function renderSyn() { synSizeEl.textContent = String(node.synaptome.size); }
renderSyn();
const synTimer = setInterval(renderSyn, 250);

// -- 5. Subscribe (deferred until mesh stabilises) ---------------
// Gate: synaptome.size >= READY_SYNAPSE_COUNT (or timeout).
const READY_SYNAPSE_COUNT = 4;   // bridge + ~3 mesh peers
const READY_TIMEOUT_MS    = 10000;

const receivedEl = document.getElementById('receivedLog');
let receivedCount = 0;
```

```
async function waitForMeshReady() {
  const t0 = Date.now();
  while (Date.now() - t0 < READY_TIMEOUT_MS) {
    if (node.synaptome.size >= READY_SYNAPSE_COUNT)
      return node.synaptome.size;
    await new Promise(r => setTimeout(r, 200));
  }
  return node.synaptome.size;
}

setStatus('connecting', 'waiting for mesh to stabilise...');
const finalSynSize = await waitForMeshReady();
log('mesh ready', { synaptomeSize: finalSynSize });

const sub = await peer.sub(TOPIC, (envelope) => {
  // Show every message we subscribe to, including our own publishes
  // -- the kernel loops them back through K-closest, and a
  // subscriber on a topic genuinely received the message regardless
  // of who signed it.  Self-published envelopes are tagged "(you)".
  const fromMe = envelope.signerPubkey === identity.pubkeyHex;
  if (receivedCount === 0) receivedEl.textContent = '';
  receivedCount++;
  const ts = new Date().toLocaleTimeString();
  receivedEl.innerHTML +=
    '<span class="arrived">[' + ts + ']</span> ' +
    escapeHtml(envelope.message) +
    (fromMe ? '<span class="self"> (you)</span>' : '') + '\n' +
    '  msgId:  ' + envelope.msgId + '\n' +
    '  signer: ' + (envelope.signerPubkey || '') + '\n\n';
  receivedEl.scrollTop = receivedEl.scrollHeight;
}, { publisher, since: 'all' });
document.getElementById('subTopicId').textContent = sub.topicId;
log('subscribed',
    { topicId: sub.topicId, synaptomeSize: node.synaptome.size });
setStatus('connected',
  `subscribed -- topicId=${sub.topicId.slice(0, 8)}...` +
  ` -- synaptome=${node.synaptome.size}`);

// -- 6. Publish button ------------------------------------------
const pubBtn    = document.getElementById('pubBtn');
const msgInput  = document.getElementById('msgInput');
const nameInput = document.getElementById('nameInput');
pubBtn.disabled = false;

const lastMsgIdEl = document.getElementById('lastMsgId');
async function publishOnce() {
  const name = (nameInput.value || 'Alice').trim();
  const msg  = (msgInput.value  || '').trim();
  if (!msg) { msgInput.focus(); return; }
  const wireMsg = name + ': ' + msg;
  pubBtn.disabled = true;
  try {
    const msgId = await peer.pub(TOPIC, wireMsg, { publisher });
    lastMsgIdEl.textContent = msgId;
    log('published', {
      wireMsg, topicId, msgId,
      synaptomeSize: node.synaptome.size,
    });
  } catch (err) {
    lastMsgIdEl.textContent = '(error: ' + err.message + ')';
    log('publish error', err.message);
  }
  pubBtn.disabled = false;
  msgInput.focus();
  msgInput.select();
}
pubBtn.addEventListener('click', publishOnce);
msgInput.addEventListener('keydown', (e) => {
```

```
      if (e.key === 'Enter' && !pubBtn.disabled) publishOnce();
    });

    function escapeHtml(s) {
      return String(s)
        .replaceAll('&', '&amp;')
        .replaceAll('<', '&lt;')
        .replaceAll('>', '&gt;');
    }
  </script>
</body>
</html>
```